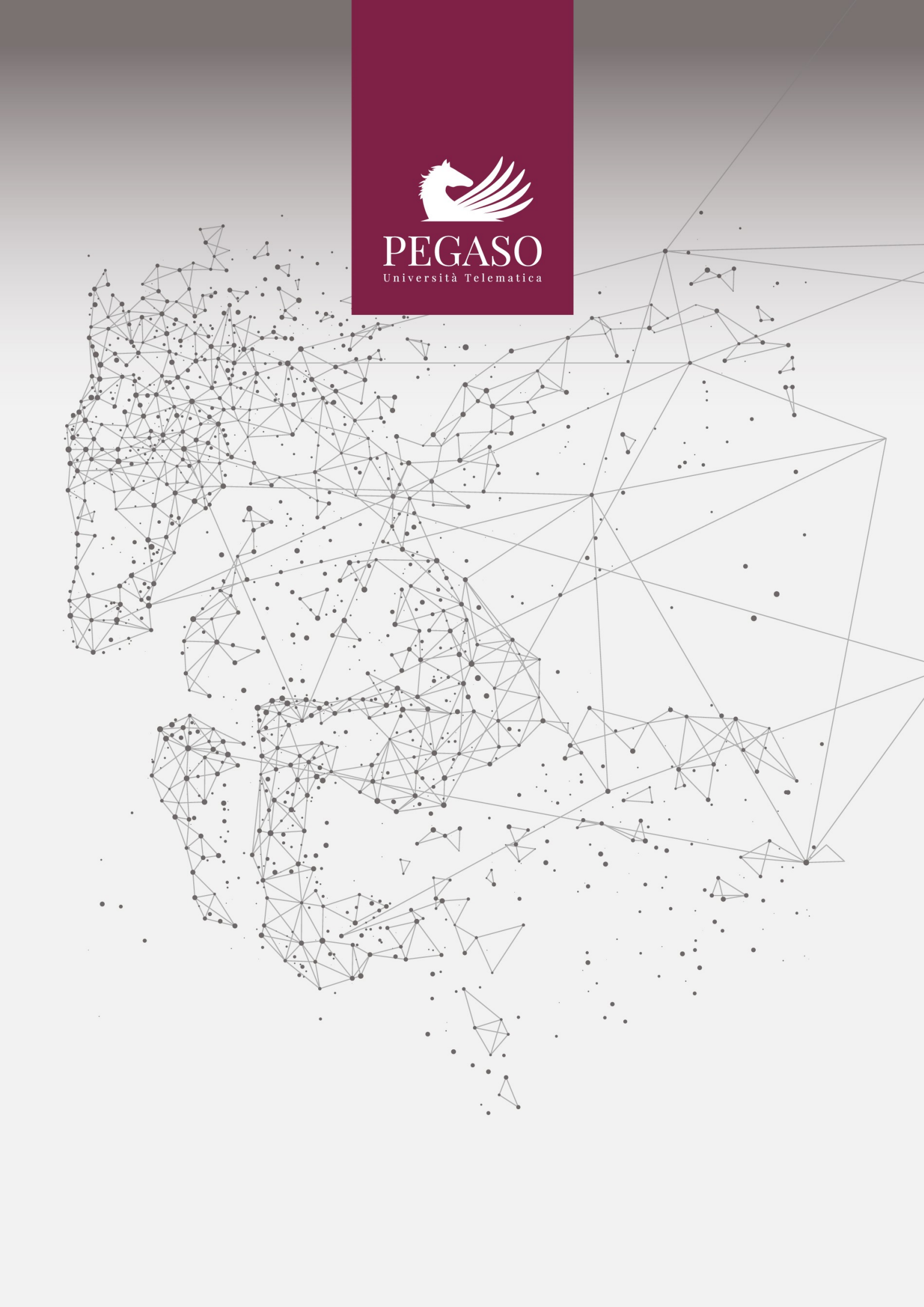

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. CALCOLO DELLE MISURE LESSICOMETRICHE DI UN TESTO....4
3. ANALISI DEL PROBLEMA E PROGETTAZIONE DELLA SOLUZIONE....5
4. FORMALIZZAZIONE DELLA SOLUZIONE IN PYTHON....9
BIBLIOGRAFIA....17

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

Questa dispensa di accompagnamento alla video-lezione illustra una esercitazione che approfondisce i concetti, le proprietà e i metodi delle stringhe del linguaggio Python. In particolare, l'esercitazione proposta vuole fornire una maggiore consapevolezza delle potenzialità e della varietà dei metodi delle stringhe offerte dal linguaggio. In particolare, nell'esercitazione verrà sviluppato passo passo un programma che calcola alcune misure lessicometriche di un testo in lingua italiana. Il programma legge il testo da un file, lo suddivide in entità lessico grafiche di interesse e ne calcola alcune misure come il numero di frasi, la lunghezza in parole, la grandezza del vocabolario. Nella realizzazione del programma oltre alle stringhe verranno utilizzati altri elementi del linguaggio Python come le liste e i dizionari per esercitarsi nell'uso di questi tipi di dati che aiutano a risolvere problematiche ricorrenti nel trattamento dei testi. Operazioni come la normalizzazione del testo, la tokenizzazione e la misurazione di unità testuali, vengono facilmente svolte usando metodi peculiari delle stringhe, liste e dizionari offerti dal linguaggio Python. L'esercitazione è costituita da tre paragrafi che seguono le tipiche fasi di sviluppo di un prodotto/sistema software. Nel primo paragrafo vengono illustrate e definite le misure lessicometriche di interesse. Nel secondo viene fatta un'analisi del problema e viene proposta una progettazione della soluzione. In questa fase seguendo l'approccio top-down verrà definita una decomposizione funzionale della struttura del programma. Infine, nel terzo paragrafo viene illustrata la formalizzazione della soluzione in Python e, come esempio, il risultato di una esecuzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Calcolo delle misure lessicometriche di un testo

In linguistica computazionale, il calcolo di misure lessicometriche viene utilizzato ampiamente per effettuare analisi automatiche del testo a livello lessicale (a livello delle parole). Questa esercitazione illustra come realizzare un semplice programma in linguaggio Python che:

1. legge un testo in italiano contenuto in un file;
2. ne analizza il contenuto;
3. calcola e produce a video un esempio di misure lessicometriche del testo proposto.

Di seguito vengono descritte le misure lessicometriche che il programma deve produrre come risultato dell'analisi lessicale del testo:

1. il ***numero di frasi*** (la regola da utilizzare per l'identificazione di una frase è "punto – spazio" oppure "punto – \n");
2. la ***lunghezza in parole*** del corpus **N**, definita dal numero totale di occorrenze in token che lo compongono;
3. la ***grandezza del vocabolario*** del testo/corpus **V**, definita dal numero totale di forme grafiche (o token) diverse (escludendo la punteggiatura);
4. la ***ricchezza lessicale*** del corpus, definita dal rapporto **V**/**N** dove i valori ottenuti da questo rapporto oscillano tra 0 e 1. Valori vicini allo 0 indicano che il vocabolario del testo non è molto grande e, quindi, non è molto vario;
5. la ***lunghezza media delle frasi*** in termini di token;
6. la ***lunghezza media dei token*** in termini di caratteri (escludendo la punteggiatura);
7. la ***frequenza assoluta*** o *frequenza di occorrenza* (*fa*), definita come il numero totale di occorrenze di uno stesso token all'interno del testo;
8. la ***frequenza relativa*** (*fr*) che indica il rapporto tra la frequenza di occorrenza di un token sul numero totale delle occorrenze rilevate nel corpus: *fr* = *fa*/*N.* La frequenza relativa di una parola può essere intesa come indice di probabilità. Se tale frequenza è alta, ci saranno alte probabilità che la parola in questione venga utilizzata in testi della stessa tipologia semantica;
9. la stampa della ***lista dei token*** (escludendo la punteggiatura) ordinati in ordine di *frequenza assoluta* decrescente, indicando anche la *frequenza relativa.*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Analisi del problema e progettazione della soluzione

In questo paragrafo verrà illustrata la fase di analisi del problema e la progettazione della soluzione.

Seguendo l'approccio top-down è possibile individuare facilmente nel problema una serie di fasi procedurali distinte che ci suggeriscono di decomporlo in 4 sottoproblemi più semplici da analizzare e risolvere:

1. caricamento del testo da file;
2. pre-trattamento e normalizzazione;
3. tokenizzazione e calcolo delle misure lessicometriche;
4. stampa a video dei risultati.

Come è facilmente intuibile, a ciascuno di questi sottoproblemi corrisponde una elaborazione distinta e ben definita. È, quindi, possibile progettare il programma seguendo un approccio funzionale: una funzione per ciascuno dei sottoproblemi. La funzione *main()* eseguita automaticamente all'attivazione del programma avrà, invece, il compito di coordinare in maniera opportuna l'attivazione delle quattro funzioni ausiliarie.

Per ciascuna funzione, si riporta una sorta di "carta di identità" della funzione stessa che indica nome, input, output ed elaborazione operata.

Si procede col progettare la funzione che realizza il *caricamento del testo*:

- *Nome della funzione*: **carica_testo(filename)**;
- *Input*: nome del file contenente il testo da analizzare;
- *Output*: il contenuto del file in un'unica stringa di testo;
- *Elaborazione operata dalla funzione*:
    1. apre in lettura il file e crea una variabile stringa vuota;
    2. itera su tutte le righe di testo contenute nel file:
        i. concatena alla variabile stringa la riga letta;
    3. chiude il file;
    4. restituisce la variabile stringa valorizzata con l'intero contenuto del file.

La progettazione della funzione che realizza il *pre-trattamento e normalizzazione del testo* prevede:

- *Nome della funzione*: **normalizza_testo(testo)**;
- *Input*: una stringa di testo valorizzata con il contenuto del file;
- *Output*: una stringa valorizzata con il contenuto del file normalizzato;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- *Elaborazione operata dalla funzione*: l'intervento di trasformazione del testo consiste di due elaborazioni distinte:
  1. *normalizzazione delle maiuscole* - il testo è case-sensitive, di conseguenza una stessa parola scritta sia in maiuscolo che in minuscolo o con la sola iniziale maiuscola non verrebbe riconosciuta come la stessa. Tale problema viene risolto eseguendo una conversione di tutti i caratteri in minuscolo;
  2. *trasformazione dei caratteri di punteggiatura* -apostrofi, doppie virgolette, TAB e fine riga sono sostituiti con il carattere spazio per rendere più semplice il trattamento successivo del testo.

La progettazione della funzione che realizza la *tokenizzazione e calcolo delle misure lessicometriche* prevede:

- *Nome della funzione*: **tokenizza_e_misura(testo)**;
- *Input*: stringa di testo formata dal contenuto del file normalizzato;
- *Output*: è costituito da 3 valori ed un oggetto:
  1. il ***numero di frasi*** di cui è composto il testo;
  2. la ***lunghezza in parole*** del testo tokenizzato (N);
  3. la ***lunghezza media delle frasi;***
  4. il ***dizionario dei token***: ciascun elemento del dizionario (con chiave il token stesso) conterrà le seguenti informazioni:
     1) *frequenza assoluta;*
     2) *frequenza relativa;*
     3) *lunghezza in caratteri;*
- *Elaborazione operata dalla funzione*: l'elaborazione consiste essenzialmente di tre fasi procedurali in cascata:
  1. *trasformazione del testo* da stringa ad una ***lista di frasi*** utilizzando per l'identificazione di una frase la sola regola "punto – spazio" (nella normalizzazione del testo la sequenza "punto – \n" è stata sostituisca con un "punto – spazio");
  2. *tokenizzazione delle frasi e costruzione del dizionario* dei token:
     - la chiave del dizionario è costituita dallo stesso token, mentre il valore è formato da una lista composta dalla coppia di valori (*<int_fa>, <float_fr>*);
     - il primo elemento della lista è la *frequenza assoluta,* mentre il secondo è la *frequenza relativa* del token;
     - durante questa fase viene calcolata solo la *fa,* mentre la *fr* verrà calcolata nella fase successiva;

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. calcolo della *frequenza relativa* per tutti i token del dizionario.

La progettazione della funzione che realizza la *stampa a video dei risultati* prevede:

- *Nome della funzione*: **print_misure(nfrasi, N, lmedia_f, dtoken)**;
- *Input*: è formato da una lista di 4 parametri, 3 valori ed un oggetto:
  1. *nfrasi*: numero di frasi contenute nel testo;
  2. *N*: la lunghezza in parole/token del testo;
  3. *l_media_f*: la lunghezza media delle frasi;
  4. *dtoken*: il dizionario dei token prodotto dalla funzione di tokenizzazione;
- *Output:* stampa a video delle misure lessicometriche calcolate;
- *Elaborazione operata dalla funzione*: l'elaborazione consiste essenzialmente di quattro fasi procedurali in cascata:
  1. stampa dei parametri globali: *numero di frasi, lunghezza media delle frasi, lunghezza in parole, grandezza del vocabolario, ricchezza lessicale;*
  2. scansione del dizionario dei token e costruzione di una lista di liste formata dalle triple di valori (*<int_fa>, <str_token>, <float_fr>*). Tale operazione è necessaria per poter ordinare la lista nella fase successiva;
  3. ordinamento in ordine di *frequenza assoluta* decrescente della lista;
  4. iterazione sugli elementi della lista e stampa a video dei valori.

  La funzione *main()*, come già anticipato, viene attivata al momento dell'esecuzione del programma.

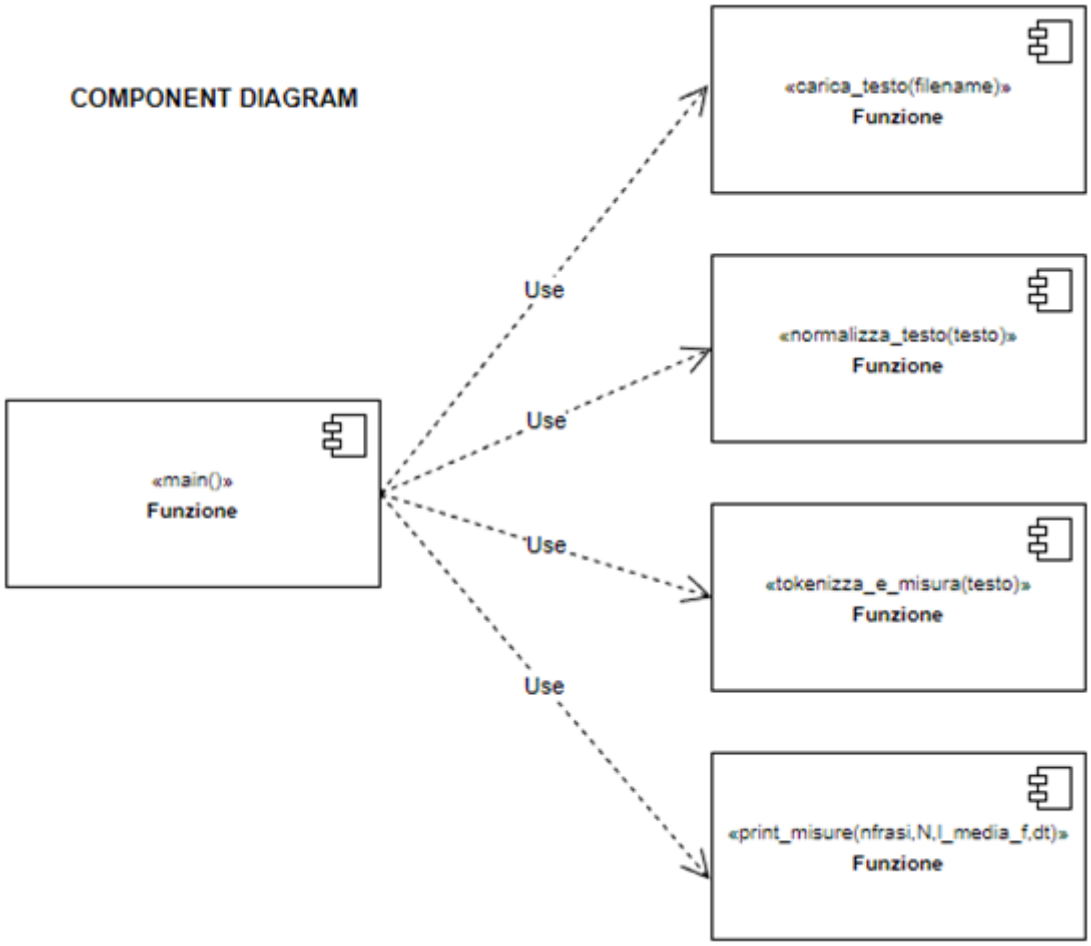
  Essa coordina l'attivazione delle funzioni ausiliarie seguendo le fasi procedurali corrette per la risoluzione del problema nel suo complesso. La sua progettazione prevede:

- *Nome della funzione*: **main()**;
- *Input*: nome del file con il testo da analizzare, viene richiesto mediante la funzione *input()*;
- *Output*: visualizzazione a video delle misure lessicometriche calcolate sul testo contenuto nel file;
- *Elaborazione operata dalla funzione*: è essenzialmente una sequenza di chiamate alle funzioni ausiliarie illustrate sopra:
  1. chiede all'utente il nome del file;
  2. chiama la funzione *carica_testo(filename);*
  3. chiama la funzione *normalizza_testo(testo);*
  4. chiama la funzione *tokenizza_e_misura(testo);*
  5. chiama la funzione *print_misure(nfrasi,N,l_media_f,dt)*

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, si completa la progettazione delle funzioni con una rappresentazione in forma grafica della struttura complessiva del programma attraverso un diagramma UML delle componenti riportato nella figura seguente.



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Formalizzazione della soluzione in Python

In questo paragrafo verrà illustrata passo passo la formalizzazione di tutte le funzioni definite nel paragrafo precedente. Si illustrerà, inoltre, a titolo di esempio, il risultato di una esecuzione del programma con un file di testo sufficientemente significativo. Nell'illustrazione della formalizzazione della soluzione verrà seguito lo stesso ordine del paragrafo precede: prima la formalizzazione delle funzioni ausiliarie e poi quella della funzione *main()*.

Il codice seguente riporta la formalizzazione della funzione **carica_testo(filename).** In particolare, nel codice, il file viene aperto in lettura usando la funzione *open(). S*i usa la variabile *f* per identificare il file.

Viene, inoltre, creata una variabile stringa *corpo* vuota*,* per contenere il testo.

Un ciclo iterativo *for..in* viene utilizzato per leggere tutte le righe di testo contenute nel file. La riga letta viene aggiunta al contenuto della variabile *corpo* con l'operatore di concatenazione '+'. Terminato il ciclo di lettura, il file viene chiuso con la funzione *close()* e la funzione termina restituendo la variabile *corpo.*

```
def carica_testo(filename):
    #Apertura del file
    f = open(filename,'r')

    #creazione stringa con il testo del file
    corpo = ""
    for r in f:
        corpo += r
    f.close()
    return corpo
```

Il codice seguente riporta la formalizzazione della funzione ***normalizza_testo(testo)***. In particolare, nel codice, viene dapprima effettuata la *normalizzazione delle maiuscole*: tutti i caratteri del testo (parametro di input alla funzione) vengono convertiti in minuscolo utilizzando il metodo *lower()* delle stringhe. Successivamente, si opera la *trasformazione dei caratteri di punteggiatura*: i caratteri di punteggiatura sono convertiti nel carattere spazio; la lista dei caratteri da convertire sono contenuti nella variabile stringa *punteggiatura*.

Per inizializzare la variabile stringa *punteggiatura* viene utilizzata la forma di concatenare due stringhe distinte per poter includere nella prima l'apice singolo e nella seconda il doppio apice. Viene, poi, utilizzato un ciclo iterativo *for … in* per sostituire nel testo uno per volta i caratteri con lo spazio. La

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

sostituzione del carattere viene effettuata con il metodo *replace()* delle stringhe. Infine, la funzione termina restituendo il testo normalizzato.

```
#Normalizzazione delle maiuscole
    testo=testo.lower()

    #Trasformazione della punteggiatura
    punteggiatura="`'" + `-:[](),;!?/\n\t"`
    for c in punteggiatura:
        testo=testo.replace(c, " ")
    return testo
```

Il codice seguente riporta la prima parte della formalizzazione della funzione **tokenizza_e_misura(testo).** In particolare, nel codice, il testo viene trasformato in una lista di frasi con il metodo *split()* delle stringhe (si usa la sola regola "punto – spazio"). Vengono, inoltre, create le variabili necessarie per il calcolo delle misure.

```
def tokenizza_e_misura(testo):
    #Trasformazione in una lista di frasi
    lfrasi = testo.split(". ")
    nf = len(lfrasi)
    #accumula la lunghezza delle frasi
    lf = 0
    #conteggio dei token/parole
    nt = 0
    #Dizionario dei token, struttura:
    #   {token1:(fa1, fr1),token2:(fa2, fr2),....}
    dtoken = {}
```

Nel codice seguente, la funzione *tokenizza_e_misura()* continua con la tokenizzazione delle frasi che viene realizzata con due cicli *for … in* annidati. Il ciclo più *esterno* itera sulla lista delle frasi, elimina gli spazi ad inizio e fine frase, accumula la lunghezza della frase e costruisce la lista dei token che formano la frase.

Il ciclo *for … in* interno itera sulla lista dei token: se il token non è presente nel dizionario lo aggiunge con i valori iniziali di *fa* e *fr*, altrimenti incrementa il valore di fa conteggiando così le occorrenze del token. Esso conteggia anche il numero totale dei token del testo nella variabile *nt*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
#Tokenizzazione delle frasi nel dizionario
    for f in lfrasi:
        f = f.strip()
        lf += len(f)
        ltoken = f.split()
        for t in ltoken:
            if t not in dtoken:
                dtoken[t] = [1,0.0]
            else:
                dtoken[t][0] += 1
            nt += 1
```

Al termine dei due cicli è stato creato il dizionario di tutti i token distinti contenuti nel testo, ma manca ancora il calcolo della frequenza relativa. Nella terza e ultima parte del codice della funzione *tokenizza_e_misura()* riportata di seguito, si procede con tale calcolo che viene effettuato con un ciclo *for … in*. Il ciclo itera su tutte le chiavi del dizionario ed aggiorna il campo *fr* avendo già calcolato il valore delle occorrenze del token (*dtoken[t][0]*) e il numero totale dei token *nt.* L'intera funzione termina restituendo il numero totale delle frasi (variabile *nf),* il numero totale dei token (variabile *nt),* la lunghezza media delle frasi (variabile *lf/nf),* l'oggetto dizionario dei token (variabile *dtoken).*

```
#calcolo della frequenza relativa
for t in dtoken:
     dtoken[t][1] = float(dtoken[t][0])/float(nt)
#Output della funzione
return nf, nt, lf/nf, dtoken
```

Ai fini di una maggiore chiarezza, di seguito si riporta il codice completo della funzione *tokenizza_e_misura()*.

```
def tokenizza_e_misura(testo):
    #Trasformazione in una lista di frasi
    lfrasi = testo.split(". ")
    nf = len(lfrasi)
    #accumula la lunghezza delle frasi
    lf = 0
    #conteggio dei token/parole
    nt = 0
    #Dizionario dei token, struttura:
    #   {token1:(fa1, fr1),token2:(fa2, fr2),....}
    dtoken = {}
    #Tokenizzazione delle frasi nel dizionario
    for f in lfrasi:
        f = f.strip()
        lf += len(f)
        ltoken = f.split()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
        for t in ltoken:
            if t not in dtoken:
                dtoken[t] = [1,0.0]
            else:
                dtoken[t][0] += 1
            nt += 1
    #calcolo della frequenza relativa
    for t in dtoken:
        dtoken[t][1] = float(dtoken[t][0])/float(nt)
    #Output della funzione
    return nf, nt, lf/nf, dtoken
```

Il codice seguente riporta la prima parte della formalizzazione della funzione **print_misure(nfrasi,N,lmedia_f,dtoken).** In particolare, nel codice, la stampa dei parametri globali viene formalizzata con una sequenza di 5 istruzioni *print().* Nei casi di valori *float* è stata utilizzata la formattazione a 6 cifre decimali con lo zero prima del punto.

```
def print_misure(nfrasi,N,lmedia_f,dtoken):
    #numero di token distinti nel testo
    V=len(dtoken)
    print("Numero di frasi: ", nfrasi)
    print("Lunghezza media delle frasi: %0.6f" % lmedia_f)
    print("Lunghezza in parole  N: ", N)
    print("Grandezza del vocabolario V: ", V)
    print("Ricchezza lessicale  V/N: %0.6f" % (V/N))
```

Poiché viene richiesta una stampa ordinata per frequenza, per poter utilizzare i metodi di ordinamento, nel codice seguente della funzione *print_misure()* viene creata una lista a partire dal dizionario dei token. In particolare, viene costruita una lista di liste. Ogni elemento è formato da una tripla di valori (*<int_fa>, <str_token>, <float_fr>*). L'operazione viene effettuata con un ciclo *for … in* che itera su tutte le chiavi del dizionario: estrae i valori di ciascun elemento e aggiunge alla lista la tripla di valori inserendo in prima posizione il campo *fa* proprio per l'ordinamento richiesto.

```
#Costruisce dal dizionario una lista per ordinarla
l=[]
for k in dtoken:
      fa = int(dtoken[k][0])
           fr = dtoken[k][1]
           l = l + [(fa,k,fr)]
```

Successivamente, come riportato nel codice seguente, la funzione *print_misure()* esegue l'ordinamento in ordine di frequenza assoluta decrescente della lista, ordinamento che viene realizzato

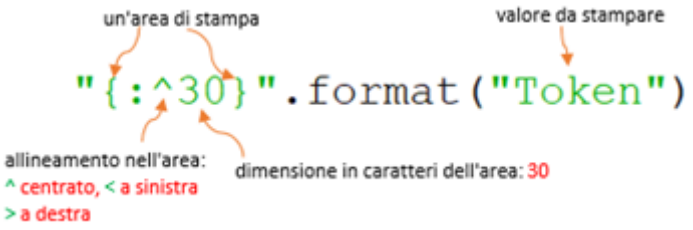
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

mediante il metodo sort() delle liste con parametro reverse=True per pilotare l'ordinamento in ordine inverso.

```
#Ordinamento della lista
l.sort(reverse=True)
```

Per una migliore leggibilità del risultato, la funzione *print_misure()* stampa a video una riga di testata come legenda delle colonne. Inoltre, la formattazione e l'incolonnamento vengono realizzati utilizzando il metodo *format()* applicato ad una stringa che fornisce le specifiche di formattazione come illustrato nella figura seguente. In particolare, l'allineamento nell'area è centrato, mentre la dimensione dell'area è pari a 30 caratteri.



Di seguito si riporta il codice della funzione *print_misure()* per la stampa a video dell'intestazione della lista.

```
#Stampa a video la testata della lista
    print("{:^30}".format("Token"),\
          "{:^4}".format("fa"),\
          "{:^8}".format("fr"),\
          "{:^4}".format("len"),sep="")
```

Infine, la funzione *print_misure()* procede con la stampa delle misure lessicometriche relative ai token del testo analizzato. La stampa viene realizzata mediante un ciclo *for … in* che itera su tutti gli elementi della lista. Per ciascun elemento della lista:

- stampa il *valore del token*;
- il valore *fa*, numero di occorrenze nel testo;
- il valore *fr*, frequenza relativa;
- la *lunghezza in caratteri* del token.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta l'ultima porzione del codice della funzione *print_misure()* che stampa a video la lista dei token del testo analizzato.

```
#Stampa a video la lista
    for e in l:
        print("{:<30}".format(e[1]),\
              "{:^4}".format(e[0]),\
              "{:^0.6f}".format(e[2]),\
              "{:^4}".format(len(e[1])),sep="")
```

Ai fini di una maggiore chiarezza, di seguito si riporta il codice completo della funzione *print_misure()*.

```
def print_misure(nfrasi,N,lmedia_f,dtoken):
    #numero di token distinti nel testo
    V=len(dtoken)
    print("Numero di frasi: ", nfrasi)
    print("Lunghezza media delle frasi: %0.6f" % lmedia_f)
    print("Lunghezza in parole  N: ", N)
    print("Grandezza del vocabolario V: ", V)
    print("Ricchezza lessicale  V/N: %0.6f" % (V/N))
    #Costruisce dal dizionario una lista per ordinarla
    l=[]
    for k in dtoken:
        fa = int(dtoken[k][0])
        fr = dtoken[k][1]
        l = l + [(fa,k,fr)]
    #Ordinamento della lista
    l.sort(reverse=True)

    #Stampa a video la testata della lista
    print("{:^30}".format("Token"),\
          "{:^4}".format("fa"),\
          "{:^8}".format("fr"),\
          "{:^4}".format("len"),sep="")
    #Stampa a video la lista
    for e in l:
        print("{:<30}".format(e[1]),\
              "{:^4}".format(e[0]),\
              "{:^0.6f}".format(e[2]),\
              "{:^4}".format(len(e[1])),sep="")
```

Infine, il codice seguente riporta la formalizzazione della funzione **main().** Nel codice seguente, il main() utilizza la funzione input() per la richiesta del nome del file di testo. Successivamente, in sequenza vengono attivate le funzioni ausiliarie. Tra una attivazione e l'altra vengono create le variabili risultato per

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

poi essere passate come parametri in input alle funzioni successive: *carica_testo(), normalizza_testo(), tokenizza_e_misura()* e *prin_misure().*

```
def main():
    filename = input("Nome del file di testo: ")
    #Caricamento del testo
    testo = carica_testo(filename)
    #Pre-trattamento e normalizzazione
    testo = normalizza_testo(testo)

    #Tokenizzazione e calcolo delle misure lessicometriche
    nfrasi,N,l_media_f,dt = tokenizza_e_misura(testo)
    #Calcolo delle misure lessicometriche
    print_misure(nfrasi,N,l_media_f,dt)

    print("Programma TERMINATO!")
```

Completata la codifica del programma, per testare il codice Python, è stato selezionato come esempio di testo da analizzare il discorso di fine anno 2022 tenuto dal Presidente della Repubblica Italiana Sergio Mattarella. Una parte del testo, contenuto nel file "*Mattarella_31-12-2022.txt*", è illustrato nella seguente videata.

```
Care concittadine e cari concittadini,
un anno addietro, rivolgendomi a voi in questa occasione, definivo i sette
anni precedenti come impegnativi e complessi.
Lo è stato anche l'anno trascorso, così denso di eventi politici e
istituzionali di rilievo.
L'elezione del Presidente della Repubblica, con la scelta del Parlamento e dei
delegati delle Regioni che, in modo per me inatteso, mi impegna per un secondo
mandato.
Lo scioglimento anticipato delle Camere e le elezioni politiche, tenutesi, per
la prima volta, in autunno.
Il chiaro risultato elettorale ha consentito la veloce nascita del nuovo
governo, guidato, per la prima volta, da una donna.
È questa una novità di grande significato sociale e culturale, che era da
tempo matura nel nostro Paese, oggi divenuta realtà.
Nell'arco di pochi anni si sono alternate al governo pressoché tutte le forze
politiche presenti in Parlamento, in diverse coalizioni parlamentari.
Quanto avvenuto le ha poste, tutte, in tempi diversi, di fronte alla necessità
di misurarsi con le difficoltà del governare.
Riconoscere la complessità, esercitare la responsabilità delle scelte,
confrontarsi con i limiti imposti da una realtà sempre più caratterizzata da
fenomeni globali: dalla pandemia alla guerra, dalla crisi energetica a quella
alimentare, dai cambiamenti climatici ai fenomeni migratori.
La concretezza della realtà ha così convocato ciascuno alla responsabilità.
Sollecita tutti ad applicarsi all'urgenza di problemi che attendono risposte.
La nostra democrazia si è dimostrata dunque, ancora una volta, una democrazia
matura, compiuta, anche per questa esperienza, da tutti acquisita, di
rappresentare e governare un grande Paese.
```

Nella figura seguente, si riporta parte del risultato dell'elaborazione del programma sviluppato in Python con indicazione delle misure lessicometriche prodotte.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
Nome del file di testo: Mattarella_31-12-2022.txt
Numero di frasi:  117
Lunghezza media delle frasi: 101.282051
Lunghezza in parole  N:  1877
Grandezza del vocabolario V:  823
Ricchezza lessicale  V/N: 0.438466
            Token              fa     fr   len
di                             70 0.037294 2
la                             67 0.035695 2
e                              59 0.031433 1
il                             39 0.020778 2
che                            36 0.019180 3
è                              32 0.017048 1
le                             29 0.015450 2
per                            27 0.014385 3
l                              23 0.012254 1
del                            22 0.011721 3
a                              22 0.011721 1
della                          21 0.011188 5
in                             19 0.010123 2
con                            19 0.010123 3
non                            16 0.008524 3
un                             14 0.007459 2
una                            13 0.006926 3
i                              13 0.006926 1
alla                           13 0.006926 4
delle                          12 0.006393 5
nostra                         11 0.005860 6
loro                           11 0.005860 4
dei                            11 0.005860 3
al                             11 0.005860 2
ha                             10 0.005328 2
da                             10 0.005328 2
chi                            10 0.005328 3
si                             9  0.004795 2
repubblica                     9  0.004795 10
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).